Answers of Quantum Problems

Jianglin Lu
19. 12. 2019
Kazan Federal University

Problem 1. We have an undirected graph $G = (V, E)$, $|V| = n$, $|E| = m$. The Oracle have access to the adjacency matrix of the graph. You should construct a quantum algorithm that can find three vertices u, v, w such that there are edges (u, v) and (v, w). u and w can be connected or not. The algorithm should have smaller running time comparing to a classical algorithm.

Answer:

From the first row to the last row of the matrix A, we look for all the indexes of which elements that are equal to 1 in each row by using Grover's search. Assuming the set C_i contains all of these found indexes at the i-th search, we remove i from the set C_i if $i \in C_i$. Now, it is obvious that the number of $|C_i|$ means the number of different vertices that v_i connected. Therefore, we can successfully find the solutions (u, v_i, w) where $u \in C_i$ and $w \in C_i$ if $|C_i| > 1$. Since Grover's search needs $O(\sqrt{n/t_i})$ queries to find a solution [1], [4], we need $O(t_i\sqrt{n/t_i}) = O(\sqrt{t_i n})$ queries to find all the solutions at the i-th row using Grover's search, where t_i is the number of all the expected solutions at i-th iterations. Therefore, the total running time of the whole algorithm is $O\left((\sum_i \sqrt{t_i})\sqrt{n}\right)$, where $i \in \{1, \ldots, n\}$ and $\sum_i t_i \leq 2m$.

The algorithm for this problem is listed as follows:

Algorithm 1: Algorithms for Solving Problem 1

Input: The adjacency matrix A of the undirected graph $G = (V, E)$, where $V = \{v_1, \ldots, v_n\}$, $|V| = n$, and $|E| = m$.

Step1: Initialize $failure = 0$.

Step2: For $i = 1 : n$
  Find all the indexes of which elements that are equal to 1 in the i-th row of the adjacency matrix A by using Grover's search.
  Let the set C_i contains all of these found indexes. If $i \in C_i$, remove i from C_i.
  If $|C_i| > 1$,
    successfully found the solutions (u, v_i, w), where $u \in C_i$ and $w \in C_i$.
  else
    $failure = failure + 1$.
  End

Output: If $failure = n$, output not found, else output all the solutions (u, v, w).

Problem 2. There are $3n$ numbers each of them from $\{1 \ldots q\}$. It is known that all numbers can be separated to non-intersected triples such that their sum can be divided to q without reminder. Find indexes of the elements from any one such triple. The quantum algorithm should have smaller running time comparing to classical algorithm.

Answer:

Assume that the $3n$ numbers are $\{x_1^1, x_2^1, x_3^1, \ldots, x_1^n, x_2^n, x_3^n\}$. There are $N = C_{3n}^3 C_{3n-3}^3 C_{3n-6}^3 \cdots C_6^3 C_3^3$ possible solutions of this problem. We set an oracle f, where $f(i) = 1$ if the sum of each non-intersected triples $\{x_1^k, x_2^k, x_3^k\}$ in the i-th possible solution $\{x_{1_i}^1, x_{2_i}^1, x_{3_i}^1, \ldots, x_{1_i}^n, x_{2_i}^n, x_{3_i}^n\}$ can be divided to q without reminder, $k \in \{1, \ldots, n\}$, $i \in \{1, \ldots, N\}$. Then we find all the indexes $d \in \{d_1, \ldots, d_m\}$ such that $f(d) = 1$ by using Grover's search on f, where m is the number of solutions. Therefore, the solutions of the above problem are $\{x_{1_d}^1, x_{2_d}^1, x_{3_d}^1, \ldots, x_{1_d}^n, x_{2_d}^n, x_{3_d}^n\}$. Since Grover's search returns one of m solutions with equal probability after an expected number of $O(\sqrt{N/m})$ iterations [2], [4], the total running time of this algorithm is $O(m\sqrt{N/m}) = O(\sqrt{mN})$.

Problem 3. There is a boolean formula F in 3-CNF form (you can check https://en.wikipedia.org/wiki/Conjunctive_normal_form and https://math.stackexchange.com/questions/56369/can-someone-please-explain-3-cnf-for-me or google it for more information). There are n variables and m clauses, $n = O(m)$, $m = O(n)$. Find at least one assignment x for variables such that $F(x) = 1$. If there are no such assignment, then return "No solution". The algorithm should have smaller running time comparing to the classical brute force algorithm.

Answer:

Suppose that $x = \{x_1, \ldots, x_n\}$. Since there are 2^n possible solutions, the classical brute force algorithm needs $O(2^n)$ running time. By using Grover's search, we can reduce the running time to $O(\sqrt{2^n}) \approx O(1.414^n)$. First, we set $N = 2^n$

and use the oracle $g = \{g_1, \ldots, g_N\}$, where $g(i) = 1$ if the i-th possible solutions $x^i = \{x_1^i, \ldots, x_n^i\}$ satisfying $F(x^i) = 1$, $i \in \{1, \ldots N\}$. Then we use Grover's search on g. If we find the k-th position in g such that $g(k) = 1$, we obtain the solution $x^k = \{x_1^k, \ldots, x_n^k\}$ that satisfies $F(x^k) = 1$. Otherwise, we will know that there are no solutions. Obviously, the running time of this algorithm is $O(\sqrt{N}) = O(\sqrt{2^n}) \approx O(1.414^n)$.

The algorithm for this problem is listed as follows:

Algorithm 3: Quantum Algorithms for 3-CNF Problem

Input:	Boolean formula F in 3-CNF form with n variables and m clauses.
Step1:	Set $N = 2^n$.
Step2:	Set an oracle $g = \{g_1, \ldots, g_N\}$, where $g(i) = 1$ if the i-th possible solutions $x^i = \{x_1^i, \ldots, x_n^i\}$ satisfying $F(x^i) = 1$, otherwise $g(i) = 0$.
Step3:	Find one index k such that $g(k) = 1$ by using Grover's search.
Output:	If found, output the solution $x^k = \{x_1^k, \ldots, x_n^k\}$, else output "No solution".

Problem 4. There is a string S and a string T. such that $|T| = O(log|S|)$. Check is T substring of S. If yes then return index of the last position T in S. The algorithm should have smaller running time comparing to classical algorithm.

Answer:

The main idea is that we divide S into $(n-m+1)$ substrings of length m from left to right. Therefore, the size of each one of $(n-m+1)$ substrings is equal to m, which is also the size of T. Then we compare if there exists any one of $(n-m+1)$ substrings of S that is equal to T. Since the run time for comparing whether two strings with equal size m are equal is $O(\sqrt{m})$, the runnign time of the total algorithm is $O((n-m+1)\sqrt{m}) \approx O(n\sqrt{m})$.

The algorithm for this problem is listed as follows:

Algorithm 4-1: Algorithms for Pattern Matching Problem

Input:	Two strings $S = \{s_1, \ldots, s_n\}$ with size n and $T = \{t_1, \ldots, t_m\}$ with size m, where $m = O(logn)$.
Step1:	Initialize $equal = FALSE$ and $postion = 0$.
Step2:	For $i = 1 : (n-m+1)$ Set $equal$ to be the output of Algorithm 4-2 with two input arguments $\{s_i, \ldots, s_{i+m-1}\}$ and $\{t_1, \ldots, t_m\}$. If $equal = False$, break. else Set $position = i + m - 1$. return $TRUE$. return $FALSE$. End
Output:	If $postion = 0$, output $FALSE$, else output $position$.

Algorithm 4-2: Quantum Algorithms for Comparing Whether Two Strings With The Same Size Are Equal

Input:	Two strings $X = \{x_1, \ldots, x_k\}$ and $Y = \{y_1, \ldots, y_k\}$ with the same size k,
Step1:	Set oracle $f(i) = 1$ if $x_i \neq y_i$ for $\forall i \in \{1, \ldots, k\}$.
Step2:	Find q that satisfies $f(q) = 1$ by using Grover's search.
Step3:	If find, return $FALSE$, else return $TRUE$.
Output:	$FALSE$ or $TRUE$.

There are some other algorithms that can solve this problem in $O(\sqrt{nm})$ and $O(\sqrt{n} + \sqrt{m})$ time [5]. We briefly introduce the algorithm with $O(\sqrt{nm})$ time. The main idea is that we first use a deterministic oracle $g(i, j)$ that tells us if it is matched at position j or not. More precisely, $g(i, j) = 1$ iff T with left endpoint aligned with s_i doesn't match at position j. Therefore, for a particular $i \in \{1, \ldots, n\}$, if there is a $j \in \{1, \ldots, m\}$ such that $g(i, j) = 1$, we can find such j in $O(\sqrt{m})$ time by using Grover's search. Now, we use another oracle $f(i)$, where $f(i) = 0$ if Grover's search with oracle $g(i, j)$ finds a mismatch, and $f(i) = 1$ otherwise. If there is a i such that $f(i = 1)$, we can find this position in $O(\sqrt{n})$ time by using Grover's search. In summary, the total running time of this algorithm is $O(\sqrt{nm})$.

Obviously, Algorithm 4-1 and 4-2 can be improved to another faster algorithm with the running time of $O(\sqrt{nm})$. If we use the oracle $p(i)$, where $p(i) = 0$ if $f(i)$ in Algorithm 4-2 is equal to 1, and $p(i) = 1$ otherwise. Therefore, this improvement makes the algorithm 4-1 and 4-2 with $O(\sqrt{nm})$ time.

Problem 5. There is a sequence of n numbers $a_1, \ldots, a_n$. Find the longest ascending subsequence $a_{j1}, \ldots, a_{jk}$, where $ji <= j(i+1)$ and $a_{ji} <= a_{j(i+1)}$. The running time of the quantum algorithm should be at most $O(n^{1.5})$

Answer:

We use dynamic programming [12] as a base algorithm and quantum algorithm to speed-up. Assume that $dp[i]$ is the longest ascending subsequence ending with number a_i. We set an oracle f, where $f(j) = 1$ if $a_j \leq a_i$, otherwise $f(j) = 0$, $j \in \{1, \ldots, (i-1)\}$ and $i \in \{1, \ldots, n\}$. For each position i with value x_i, we find the maximum index k such that $f(k) = 1$ by using Grover's search. If found, we set $possion[i] = k$, which means that the penultimate elements of the longest ascending subsequence ending with number a_i is $a[position[i]]$. At the same time, we set $dp[i] = dp[k] + 1$, which means the longest ascending subsequence ending with number a_i has one more element a_i than the longest ascending subsequence ending with number $a[position[i]]$. If not found, we set $possion[i] = i$ and $dp[i] = 1$, which means that the longest ascending subsequence ending with number a_i has only one element, that is a_i itself. At last, we find the position m such that $dp[m]$ is the maximum of dp by using Grover's search. Obviously, the length of the longest ascending subsequence of $\{a_1, \ldots, a_n\}$ is $dp[m]$. Now, we analysis the running time of this algorithm. Finding the maximum index k such that $f(k) = 1$ by using Grover's search needs $O(\sqrt{n})$ times. Finding the position m such that $dp[m]$ is the maximum of dp by using Grover's search also needs $O(\sqrt{n})$ times. Therefore, the total running time of this algorithm is $O(n\sqrt{n} + \sqrt{n}) \approx O(n^{1.5})$.

The algorithm for this problem is listed as follows:

Algorithm 5: Quantum Algorithms for Longest Ascending Subsequnce Problem

Input: A sequence of n numbers $\{a_1, \ldots, a_n\}$.

Step1: Set $dp[i] = 1$ and $position[i] = 0$ for $\forall i \in \{1, \ldots, n\}$.

Step2: Set $position[1] = 1$, and the longest ascending subsequence LAS as a null set.

Step3: For $i = 2 : n$

  Set an oracle f, where $f(j) = 1$ if $a_j \leq a_i$, otherwise $f(j) = 0$, $j \in \{1, \ldots, (i-1)\}$.

  Find the maximum index k such that $f(k) = 1$ by using Grover's search.

  If found,

    Set $position[i] = k$ and $dp[i] = dp[k] + 1$;

  else,

    Set $position[i] = i$ and $dp[i] = 1$.

  End

Step4: Find the position m such that $dp[m]$ is the maximum of dp by using Grover's search.

Step5: $LAS = \{..., a\,[position[position[position[m]]]]\,, a[position[position[m]]], a[position[m]], a[m]\}$ with size $dp[m]$.

Output: LAS, the longest ascending subsequence.

Problem 6. We have the undirected graph $G = (V, E)$, $|V| = n$, $|E| = m$. The Oracle have access to the adjacency matrix of the graph. You should construct a quantum algorithm that can find k-clique of the graph. k is known constant. The running time should be at most $O(2^{n/2})$

Answer:

There are $N = 2^n$ possible subsets of a set of the undirected graph's vertices. We define an oracle f, where $f(i) = 1$ if the i-th subsets exists a clique of the graph G with size k, otherwise $f(i) = 0$, $i \in \{1, \ldots, N\}$ [8]. Then we use Grover's search on f. If we find the j-th location in f such that $f(j) = 1$, the k-clique of the graph G is the j-th subsets of the set of graph's vertices. Obviously, the running time of this algorithm is at most $O(\sqrt{N}) = O(2^{n/2})$.

The algorithm for this problem is listed as follows:

Algorithm 6: Quantum Algorithms for k-Clique Problem

Input: Undirected graph $G = (V, E)$, where $|V| = n$, $|E| = m$.

Step1: Set $N = 2^n$.

Step2: Set an oracle $f = \{f_1, \ldots, f_N\}$, where $f(i) = 1$ if the i-th subsets exists a clique of the graph G with size k, otherwise $f(i) = 0$.

Step3: Find one index j such that $f(j) = 1$ by using Grover's search.

Output: If found, output the j-th subsets as the k-clique of the graph, else output "Not found".

Problem 7. There is n integer numbers. You should construct an algorithm that sorts numbers. The running time should be at most $O(n\sqrt{n})$.

Answer:

The core idea to solve this problem is that we perform n times minimum finding algorithm using Grover's search. We use $\{y_1, \ldots, y_n\} \in \Re^1$ with size n to record the sorted results. Once we obtain the minimum x_k at the i-th searching, we set $y_i = x_k$ and change the value of x_k to $\hat{x}_k$, where $\hat{x}_k = x_k + \sum_{j=1}^{n} x_j$. Since Grover's search needs $O(\sqrt{n})$ queries for minimum finding [2], the running time of the total algorithm is $O(n\sqrt{n})$.

At the i-th searching, in fact, we can directly remove the minimum x_k from the set $\{x_1, \ldots, x_n\}$ rather than change the value of x_k to $\hat{x}_k$. The reason why we do this change operation is for the sake of recovery of original data, i.e., the set $\{x_1, \ldots, x_n\}$. Once we have finished the sorting process, we can recover the original data by setting $\hat{x}_k - \sum_{j=1}^{n} x_j$ for $\forall k \in \{1, \ldots, n\}$.

The algorithm for this problem is listed as follows:

Algorithm 7-1: Quantum Algorithms for Sorting Problem

Input: n unordered integer numbers $\{x_1, \ldots, x_n\}$.

Step1: Initialize $y_1 = ... = y_n = 0$.
Step2: Set $sum = \sum_{j=1}^{n} x_j$.
Step3: For $i = 1 : n$
  Find the minimum x_k by using Algorithm 7-2.
  Set $y_i = x_k$.
  Set $\hat{x}_k = x_k + sum$.
  End
Step4: Recover the original data by using $\hat{x}_k - \sum_{j=1}^{n} x_j$ for $\forall k \in \{1, \ldots, n\}$.

Output: n integers sorted in ascending order $y_1, \ldots, y_n$.

Algorithm 7-2: Quantum Algorithms for Minimum Finding Problem

Input: n unordered integer numbers $\{x_1, \ldots, x_n\}$.

Step1: Select x_i uniformly at random from $\{x_1, \ldots, x_n\}$.
Step2: Set $m = x_i$.
Step3: Repeat:
  Find $q \in \{x_1, \ldots, x_n\}$ such that $q < m$ by using Grover's search.
  If find, set $m = q$, else stop loop.

Output: m, the minimal of $\{x_1, \ldots, x_n\}$.

Problem 8. There is string $x2y$ where x, y are binary strings of length n. It is known that $\#_1(x) = v2^k$ and $\#_1(y) = w2^k$. You should construct a quantum streaming algorithm for checking equality of w mod 2 and v mod 2. The algorithm should use a single qubit.

Answer:

First, we set $\alpha = \frac{\pi}{2 \times 2^k}$. If $x_i = 1$ or $y_i = 1$, $\theta = \theta + \alpha$; If $x_i = 0$ or $y_i = 0$, $\theta = \theta$ [6], [7]. Then we have the following results:

If w mod $2 = v$ mod 2, $\theta \in \{0, \pi\}$. If w mod $2 \neq v$ mod 2, $\theta \in \{\frac{\pi}{2}, \frac{3\pi}{2}\}$.

Problem 9. You have a string s of latin letters and white spaces. You should implement "trim" procedure (that return index of first and last non space symbols). The running time should be at most $O(\sqrt{n})$.

Answer:

First, we use an oracle $f(i)$, where $f(i) = 1$ if i-th element s_i of s is a latin letter, $f(i) = 0$ otherwise. To find the first non space symbol, we find the minimal q such that $f(q) = 1$ by using Grover's search, which needs $O(\sqrt{n})$ queries. To find the last non space symbol, we find the maximum q such that $f(q) = 1$ by using Grover's search, which also needs $O(\sqrt{n})$ queries. Therefore, the total running time for the Algorithm 9 is $O(\sqrt{n})$.

The algorithm for this problem is listed as follows:

Algorithm 9: Quantum Algorithms for Solving Problem 9

Input: A string $s = \{s_1, \ldots, s_n\}$ of latin letters and white spaces with size n.

Step1: Set an oracle $f(i)$ where $f(i) = 1$ if i-th element s_i of s is a latin letter, $f(i) = 0$ otherwise.
Step2: Find the minimal q such that $f(q) = 1$ by the following steps.
  Find a position $k \in \{1, \ldots, n\}$ such that $f(k) = 1$ by using Grover's search.
  Set $\hat{k} = k$.
  Repeat:
    Find a new position $k \in \{1, \ldots, (\hat{k} - 1)\}$ such that $f(k) = 1$ by using Grover's search.
    If not found,
      set $q = \hat{k}$; break;
    else
      Set $\hat{k} = k$.
Step3: Find the maximal p such that $f(p) = 1$ by using Grover's search.
  Find a position $k \in \{1, \ldots, n\}$ such that $f(k) = 1$ by using Grover's search.
  Set $\hat{k} = k$.
  Repeat:
    Find a new position $k \in \{(\hat{k} + 1), \ldots, n\}$ such that $f(k) = 1$ by using Grover's search.
    If not found,
      set $p = \hat{k}$; break;
    else
      Set $\hat{k} = k$.

Output: The first and last non space symbols $s_q, \ldots, s_p\}$.

Problem 10. You have a string s of latin letters and white spaces. You should find number of words in the string. The quantum algorithm should work faster than classical one if number of words is much less than length of the string.

Answer:

We first set an oracle $f(i)$ in which $f(1) = 1$ if s_1 is a latin letter, $f(1) = 0$ otherwise. For $i \in \{2, \ldots, n\}$ let $f(i) = 1$ if s_i is a latin letter and s_{i-1} is a white space, $f(i) = 0$ otherwise. Then we find all the indexes of which elements that are equal to 1 in f by using Grover's search. The number of words in the string s is the number of all the found indexes. Since Grover's search returns one of t solutions with equal probability after an expected number of $O(\sqrt{n/t})$ iterations [2], [4], the total running time of this Algorithm 10 is $O(t\sqrt{n/t}) = O(\sqrt{tn})$, where t is the number of words in the string s. Obviously, the running time $O(\sqrt{tn})$ is less than the classical algorithm with $O(n)$ if number of words t is much less than the length n of the string s.

The algorithm for this problem is listed as follows:

Algorithm 10: Quantum Algorithms for Solving Problem 10

Input:	A string $s = \{s_1, \ldots, s_n\}$ of latin letters and white spaces with size n.
Step1:	Initial $num = 0$.
Step2:	Set an oracle $f(i)$ by the following rules. Let $f(1) = 1$ if s_1 is a latin letter, $f(1) = 0$ otherwise. For $i \in \{2, \ldots, n\}$ let $f(i) = 1$ if s_i is a latin letter and s_{i-1} is a white space, $f(i) = 0$ otherwise.
Step3:	Find all the indexes of which elements that are equal to 1 in f by using Grover's search.
Step4:	Set num to be equal to the number of all the found indexes.
Output:	num, the number of words in the string s.

Problem 11*. Proof the expected query complexity of minimum finding algorithm.

Answer:

As proved in [2], the running time is $O(\sqrt{n})$ to find the index of the minimum value. We give a relatively relaxed proof according to [3].

Assume the data is $\{x_1, \ldots, x_n\}$ with size n and $x_i \neq x_j$ unless $i = j$. Let x^0 be the initial value that needs to be compared before performing Grover's search and x^i be the i-th value that needs to be compared after i-th Grover's search. It is easy to know that x^0 is the $(n/2)$-th smallest value of the set $\{x_1, \ldots, x_n\}$ on average, since x^0 is selected uniformly at random from $\{x_1, \ldots, x_n\}$. Assume the set X^1 consists of all the elements $q \in \{x_1, \ldots, x_n\}$ that satisfy $q < x^0$. After the first Grover's search, we can obtain x^1 which is an element in the set X^1. As we know, the probability of any element being selected from the set X^1 is equal. That is to say, x^1 is selected uniformly at random for the set X^1. As mentioned above, x^0 is the $(n/2)$-th smallest value of the set $\{x_1, \ldots, x_n\}$ on average. As a result, x^1 is the $(n/4)$-th smallest value of the set $\{x_1, \ldots, x_n\}$ on average. Similarly, x^i is the $(n/2^{i+1})$-th smallest value of the set $\{x_1, \ldots, x_n\}$ on average.

As indicated in [2], [4], Grover's search return one of t solutions with equal probability after an expected number of $O(\sqrt{n/t})$ iterations. According to the above statements, it is expected that $k \approx 1$ in the last iteration before we find the minimal. Similarly, at the penultimate, third to last, ... , iteration, $k \approx 2$, $k \approx 4$, ... and so on. Therefore, the total number of the queries in the whole loop of Grover's search is of order $\sqrt{n} + \sqrt{n/2} + \sqrt{n/4} + \ldots = \sqrt{n}(1 + \frac{1}{\sqrt{2}} + \frac{1}{\sqrt{4}} + \ldots)$. Since the sum of $(1 + \frac{1}{\sqrt{2}} + \frac{1}{\sqrt{4}} + \ldots)$ is up to a constant, the running time of Grover's search to find the minimum is $O(\sqrt{n})$. □

Problem 12*. You have a sequence of strings $s_1, \ldots, s_n$ of length k. Sort the strings in lexicographical order. The running time should be small comparing to classical algorithms in a case of $k = o((log\ n)^4)$.

Answer:

The main idea to solve this problem is exploiting Heap sort [9] as a base algorithm and quantum algorithm for the purpose of comparing strings [10]. We introduce this method that solves this problem as follows.

First, we construct a function called $CTS(q, p, k)$ that compares two string $q = \{q_1, \ldots, q_k\}$ and $p = \{p_1, \ldots, p_k\}$ with the same size k in lexicographical order. We define an oracle f, where $f(i) = 1$ if $q_i \neq p_i$, otherwise $f(i) = 0$, $i \in \{1, \ldots, k\}$. Then we find the minimal index j such that $f(j) = 1$ by using Grover's search, which needs $O(\sqrt{k})$ running time. If no solutions are found, we return $k+1$ to facilitate the subsequent processes. To boost success probability, [10] proposed to repeat this process $3\log_2 n$ times to obtain the minimal answer with the error probability $O(\frac{1}{2^{3\log n}}) = O(\frac{1}{n^3})$.

Assume that $s_i = \{s_i^1, \ldots, s_i^k\}$, where $s_i \in \{s_1, \ldots, s_n\}$. [10] use Binary Heap [11] to sort the sequence and store indexes of strings in vertexes. We can compare vertexes v and $\hat{v}$ with corresponding strings s_i and $s_{\hat{i}}$. We set $v > \hat{v}$ if $s_i > s_{\hat{i}}$ in lexicographical order. The above function $CTS(s_i, s_{\hat{i}}, k)$ can be used to compare strings s_i and $s_{\hat{i}}$.

There are three main functions in Binary Heap as follows:

$INIT(BH)$, which initializes an empty heap;

$ADD(BH, s_i)$, which adds a vertex i to the heap;

$Get_Min_and_Delete(BH)$, which returns minimal s_i and removes it from the heap.

With the above functions, the algorithm is listed as follows:

Algorithm 12-1: Quantum Algorithms $CTS(q, p, k)$ for Two Strings Comparison

Input: Two strings $q = \{q_1, \ldots, q_k\}$ and $p = \{p_1, \ldots, p_k\}$ with the same size k.

Step1: Initial $j = k + 1$ and $result = 0$.

Step2: For $i = 1 : 3\log_2 n$

Set $\hat{j}$ is the minimal index $\hat{j}$ such that $f(\hat{j}) = 1$ by using Grover's search.

Set $j = min(j, \hat{j})$.

End

Step3: If $j = k + 1$,

$result = 0$;

else if $q_j < p_j$,

$result = -1$;

else,

$result = 1$.

Output: $result$.

Algorithm 12-2: Quantum Algorithms for Strings Sequence Sorting

Input: A sequence of strings $\{s_1, \ldots, s_n\}$ of length k.

Step1: Initial binary heap by using $INIT(BH)$ function.

Step2: Initial sorted sequence $\hat{s}$ as a null set.

Step3: For $i = 1 : n$

Add s_i into the heap by using $ADD(BH, s_i)$ function.

End

Step4: For $i = 1 : n$

$\hat{s} = \hat{s} + Get_Min_and_Delete(BH)$;

End

Output: $\hat{s}$.

Since $Get_Min_and_Delete$ and ADD functions needs to invoke CTS function $\log_2 n$ times, and the running time of CTS function is $O(\log_2 n\sqrt{k})$, the total running time of the whole algorithm is $O(n(\log n)^2\sqrt{k})$, as proved in [10].

References

[1] L.K. Grover, A fast quantum mechanical algorithm for database search, ArXiv. (1996).

[2] C. Durr, P. Hoyer, A quantum algorithm for finding the minimum, (1996) 1-2.

[3] A. Ambainis, Quantum search algorithms, Int. J. Mod. Phys. B. 23 (2005) 5727-5758.

[4] M. Boyer, G. Brassard, P. Hoyer, A. Tapp, Tight bounds on quantum searching, Fortschritte Der Phys. 46 (1998) 493-505.

[5] H. Ramesh, V. Vinay, String matching in $\tilde{O}(\sqrt{n} + \sqrt{m})$ quantum time, (2000) 1-7.

[6] K. Khadiev, Basics of quantum computing: Quantum bit, unitary transformations, measurement, Lecture 2, (2019).

[7] F. Ablayev, A. Gainutdinova, K. Khadiev, A. Yakarylmaz, Very narrow quantum OBDDs and width hierarchies for classical OBDDs, Lobachevskii J. Math. 37 (2014) 670-682.

[8] A. Boji, Quantum algorithm for finding a maximum clique in an undirected graph, J. Inf. Organ. Sci. 36 (2012) 91-98.

[9] T.H. Cormen, C.E. Leiserson, R.L. Rivest, Introduction to algorithms, Second Edition, (2001).

[10] K. Khadiev, A. Ilikaev, Quantum algorithms for the most frequently string search, intersection of two string sequences and sorting of strings problems, in: J. Pastor. Care Couns., (2019) 234-245.

[11] J.W.J. William, Algorithms 232-heapsort, Commun. ACM. 7(6) (1964) 347-349.

[12] S. Dasgupta, C.H. Papadimitriou, U. Vazirani, Algorithms, First Edition, (2006).